



Kimi K3 Deep Dive

World's first OPEN 3T-class model



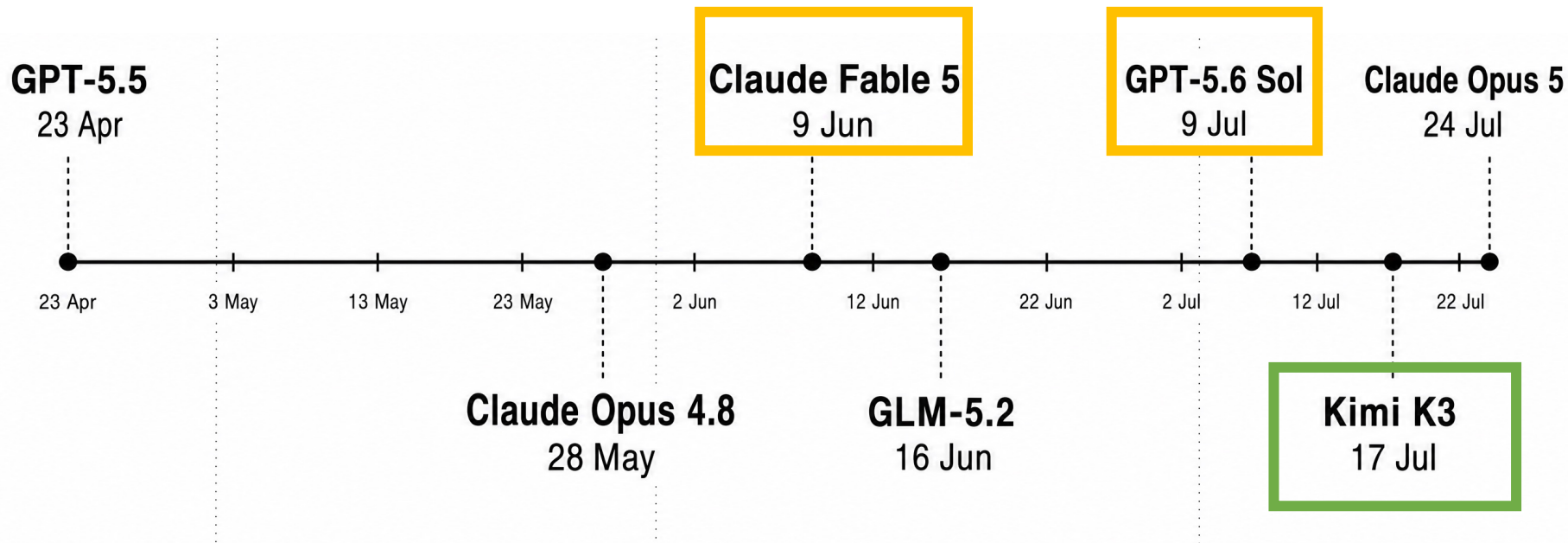
Open frontier narrows gap with closed-source models

2.8T 104B active 1M context

TOTAL PARAMETERS / PER-TOKEN ACTIVATION / CONTEXT WINDOW

K3 is the biggest open-source LLM in size and combines native multimodality, sparse activation, and long-context agent training.

LLM release timeline



General Agents

All maxed out on thinking effort: max or xhigh.

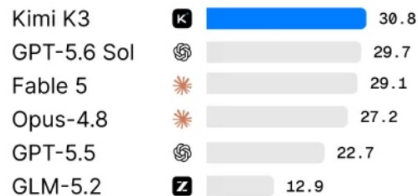
GDPval-AA v2 Elo human expert performance



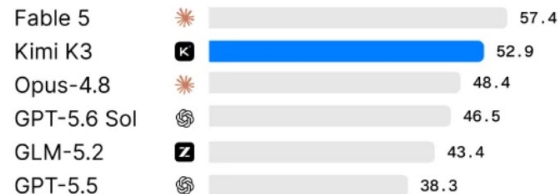
AA-Briefcase Elo



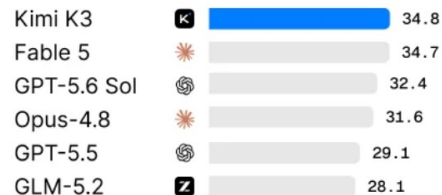
Automation Bench



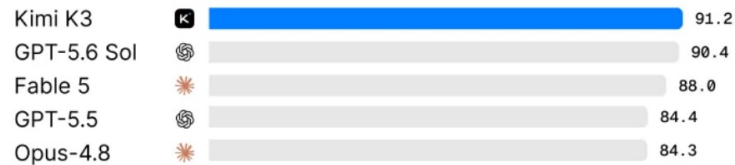
JobBench professional workflows



SpreadsheetBench 2



BrowseComp Browse the web



Coding

All maxed out on thinking effort: max or xhigh.

DeepSWE



FrontierSWE



Kimi Code Bench 2.0 (Internal)



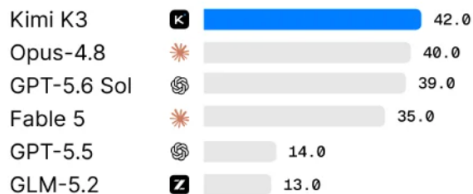
Terminal Bench 2.1



Program Bench

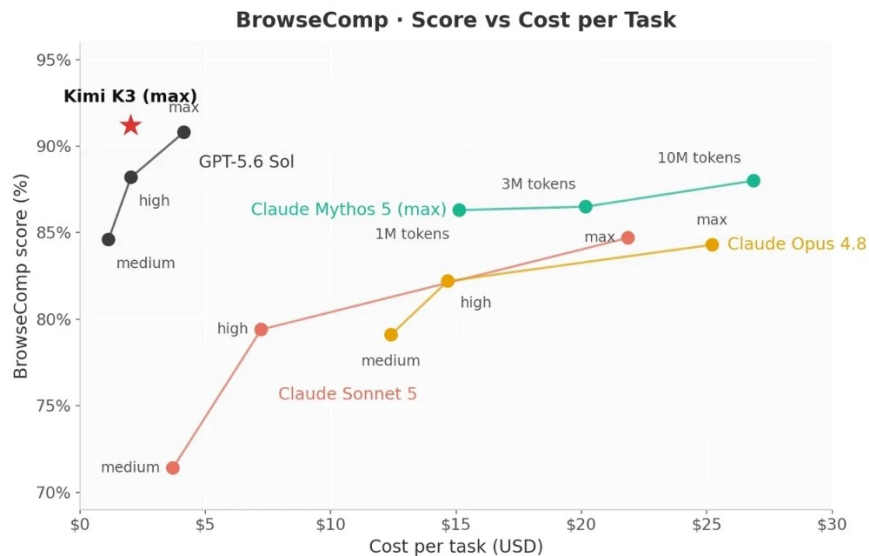
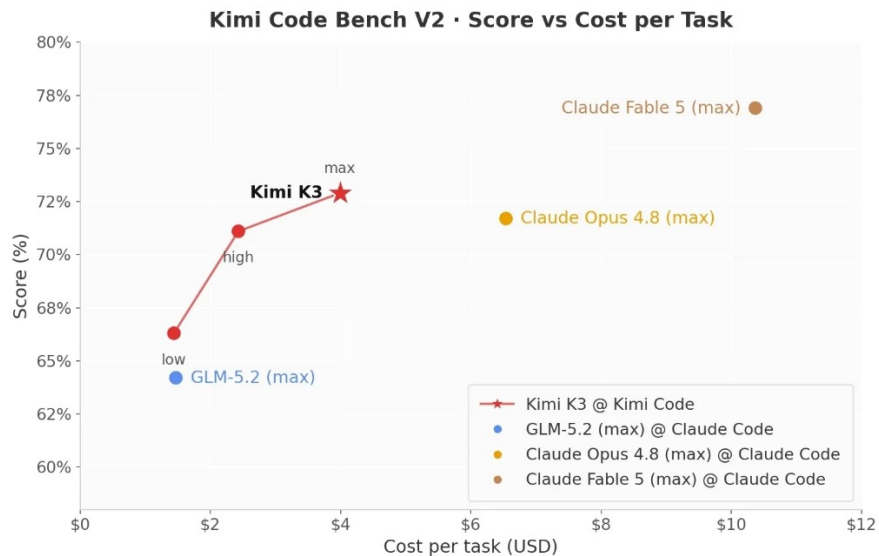


SWE Marathon multi-hour software engineering tasks



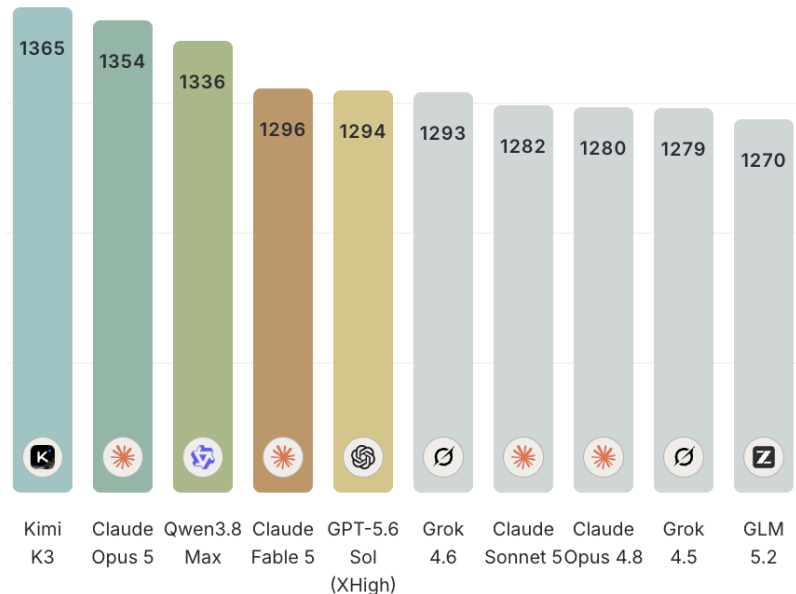
Note: All Fable 5 results are with potential fallbacks. All GPT-5.6 Sol results include potential cyberguards.

Performance v.s. Cost per task

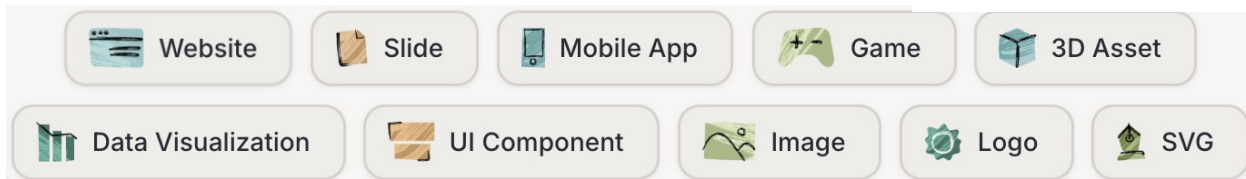
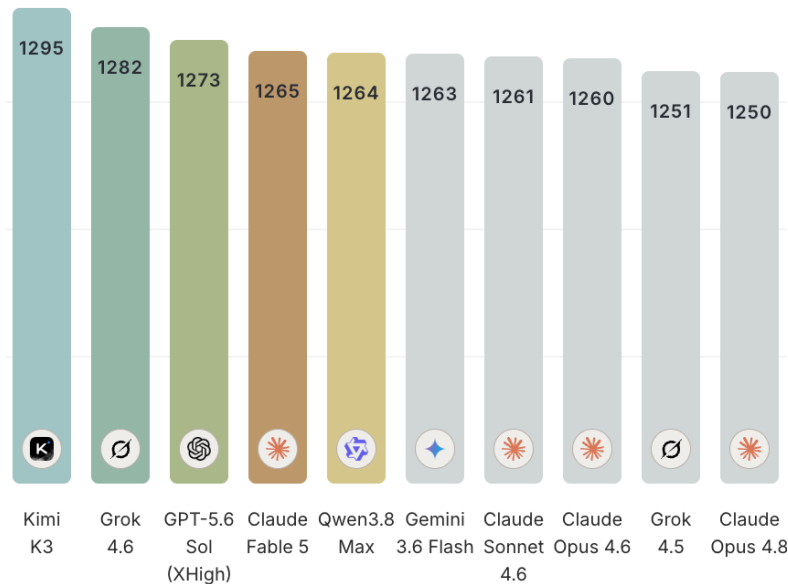


Design Arena Tournament Ranking

Full-stack Web Dev (Agentic)



mobile applications with React Native



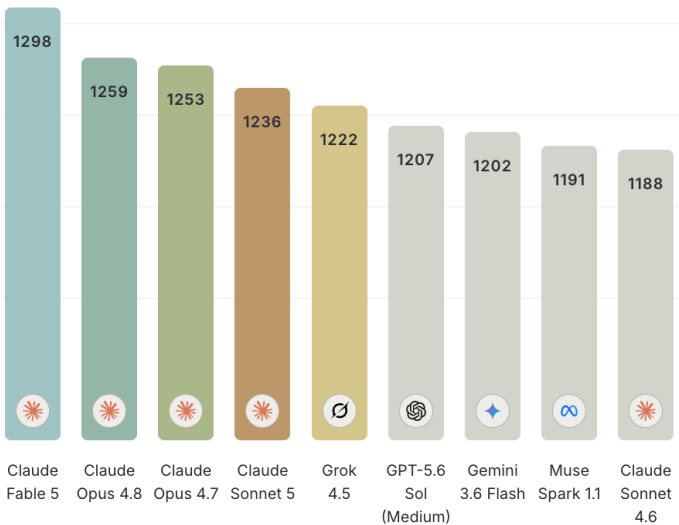
<https://www.designarena.ai/leaderboard>

Design Arena Tournament Ranking

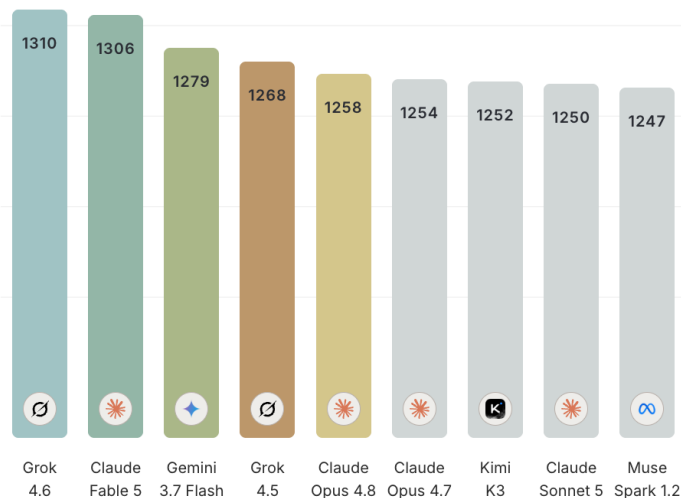
K3 not good at Game Dev.



Agentic Game Dev



Android (Kotlin)



K3 is sparse by design

K3 separates total capacity from per-token compute with **Mixture-of-experts** backbone: it routes each token through a narrow subset of a far wider expert pool.



2.8T

total parameters
capacity available across the full model

|
896 + 2

routed experts + shared experts
within a 93-layer architecture

|
16

routed experts selected per token
a 56:1 routed-expert sparsity ratio

|
104B

activated parameters per token
the compute path actually used for one token

Design implication: scale expert capacity without activating the full 2.8T model for every token.

K3 pushes three scaling dimensions

One model, three information-flow problems: **length, depth, and width.**

- **Sequence length: Kimi Delta Attention**

Nov 2025

 KIMI LINEAR:

AN EXPRESSIVE, EFFICIENT ATTENTION ARCHITECTURE

- **Model depth: Attention Residuals**

Mar 2026

 ATTENTION RESIDUALS

- **Model width: LatentMoE**

Jan 2026

 NVIDIA.

LatentMoE: Toward Optimal Accuracy per FLOP
and Parameter in Mixture of Experts

K3 Architecture

KDA: Kimi Delta Attention

LatentMoE: Latent Mixture of Experts

MLA: Multi-head latent attention

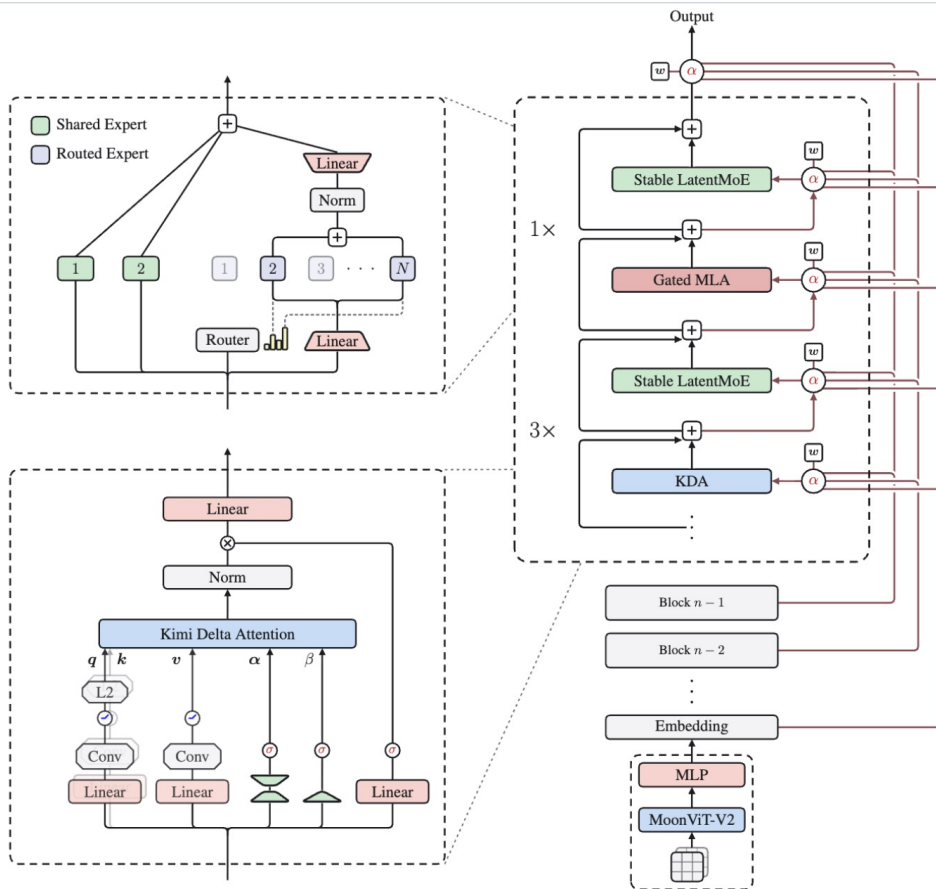
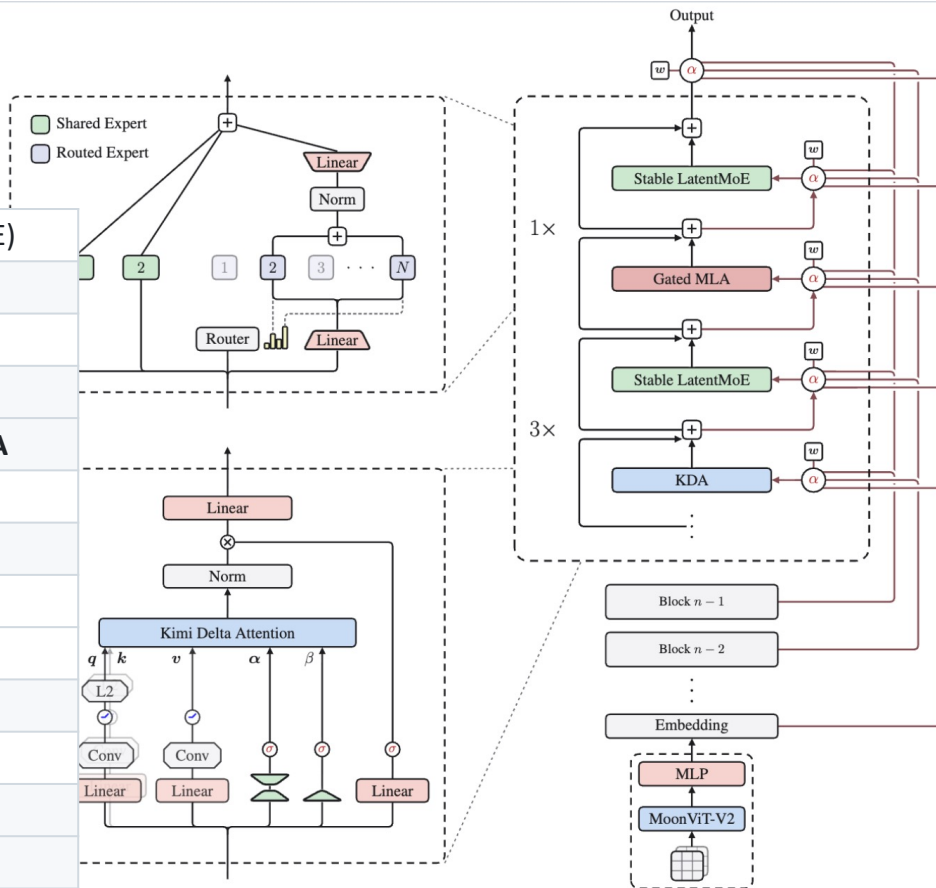


Figure 2: The Kimi K3 architecture, organized around token, channel, and layer mixing, with a native vision pathway at the input. Each block contains three Kimi Delta Attention (KDA) layers followed by one Gated MLA layer, with each attention layer paired with a Stable LatentMoE feed-forward network. Attention Residuals (AttnRes) use learned pseudo-queries ($\mathbf{w}$) to derive attention weights (α) over the embedding and preceding block outputs, enabling selective information flow across depth. **Top left:** the Stable LatentMoE module with shared and routed experts. **Bottom left:** the KDA module. **Bottom right:** the native vision pathway.

K3 Architecture

Architecture	Mixture-of-Experts (MoE)
Total Parameters	2.8T
Activated Parameters	104B
Number of Layers	93
Attention-Layer Composition	69 KDA + 24 Gated MLA
Attention Hidden Dimension	7168
Number of Attention Heads	96
Latent MoE Dimension	3584
Number of Experts	896
Selected Experts per Token	16
Number of Shared Experts	2
Vocabulary Size	160K
Vision Encoder	MoonViT-V2
Parameters of Vision Encoder	401M



K3 architecture, organized around token, channel, and layer mixing, with a native vision pathway at the input. Each block contains three Kimi Delta Attention (KDA) layers followed by one Gated MLA layer, with each attention layer paired with a Stable LatentMoE feed-forward network. Attention Residuals (AttnRes) use learned pseudo-queries (w) to derive attention weights (α) over the embedding and preceding block outputs, enabling selective information flow across depth. **Top left:** the Stable LatentMoE module with shared and routed experts. **Bottom left:** the KDA module. **Bottom right:** the native vision pathway.

KDA: From traditional attention to linear attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q \cdot K^\top}{\sqrt{d_k}} \right) \cdot V$$

Traditional formulation of the softmax attention mechanism.

$$\text{Attention}(Q, K, V) = \frac{\sum_{j=1}^N \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^N \text{sim}(Q_i, K_j)}$$

Note: $N=j$, summed up to i

General expression for attention using any similarity function.

$$\text{LinearAttention}(Q, K, V) = \frac{\phi(Q_i) (\sum_{j=1}^N \phi(K_j)^\top V_j)}{\phi(Q_i) \sum_{j=1}^N \phi(K_j)}$$

$$\phi(x) = \text{elu}(x) + 1$$

Linear attention using feature maps to approximate the softmax similarity score.

KDA: From traditional attention to linear attention

$$\text{LinearAttention}(Q, K, V) = \frac{\phi(Q_i)(\sum_{j=1}^N \phi(K_j)^\top V_j)}{\phi(Q_i) \sum_{j=1}^N \phi(K_j)} \quad \text{Note: } N=i, \text{ summed up to } i$$

Linear attention using feature maps to approximate the softmax similarity score.

$$S_i := \sum_{j \leq i} \tilde{k}_j v_j^\top \quad \text{and} \quad z_i := \sum_{j \leq i} \tilde{k}_j.$$

$$S_i = S_{i-1} + \tilde{k}_i v_i^\top,$$

$$z_i = z_{i-1} + \tilde{k}_i.$$

$$o_i = \frac{\tilde{q}_i^\top S_i}{\tilde{q}_i^\top z_i}$$

KDA: From linear attention to KDA

The KDA state update combines two token-dependent controls: α_t controls channel-wise retention of the previous memory, while β_t controls the strength of the current delta-rule write.

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\alpha_t) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top \in \mathbb{R}^{d_k \times d_v}; \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t \in \mathbb{R}^{d_v}$$

$$\begin{bmatrix} \text{blue squares} \end{bmatrix} = \left(\begin{bmatrix} \text{grey squares} \end{bmatrix} - \begin{bmatrix} \text{orange squares} \end{bmatrix} \times \begin{bmatrix} \text{orange squares} \end{bmatrix} \right) \begin{bmatrix} \text{red squares} \end{bmatrix} \begin{bmatrix} \text{blue squares} \end{bmatrix} + \begin{bmatrix} \text{orange squares} \end{bmatrix} \times \begin{bmatrix} \text{orange squares} \end{bmatrix} \quad \Bigg| \quad \begin{bmatrix} \text{orange squares} \end{bmatrix} = \begin{bmatrix} \text{blue squares} \end{bmatrix} \begin{bmatrix} \text{orange squares} \end{bmatrix}$$

State update intuition: retain selected memory + a controlled key–value correction

KDA: Prefilling & Decoding speed

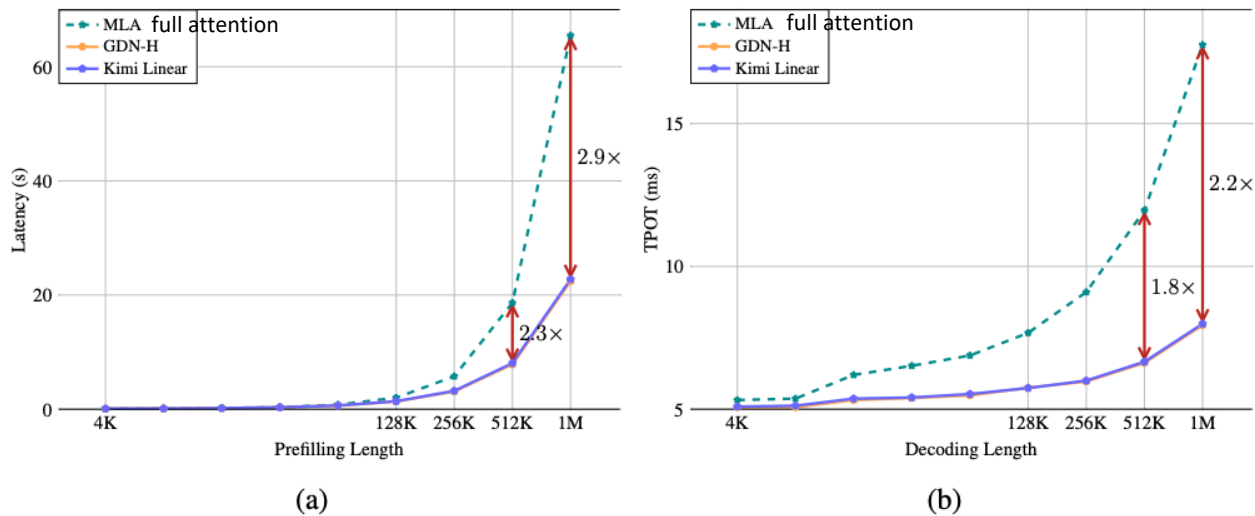
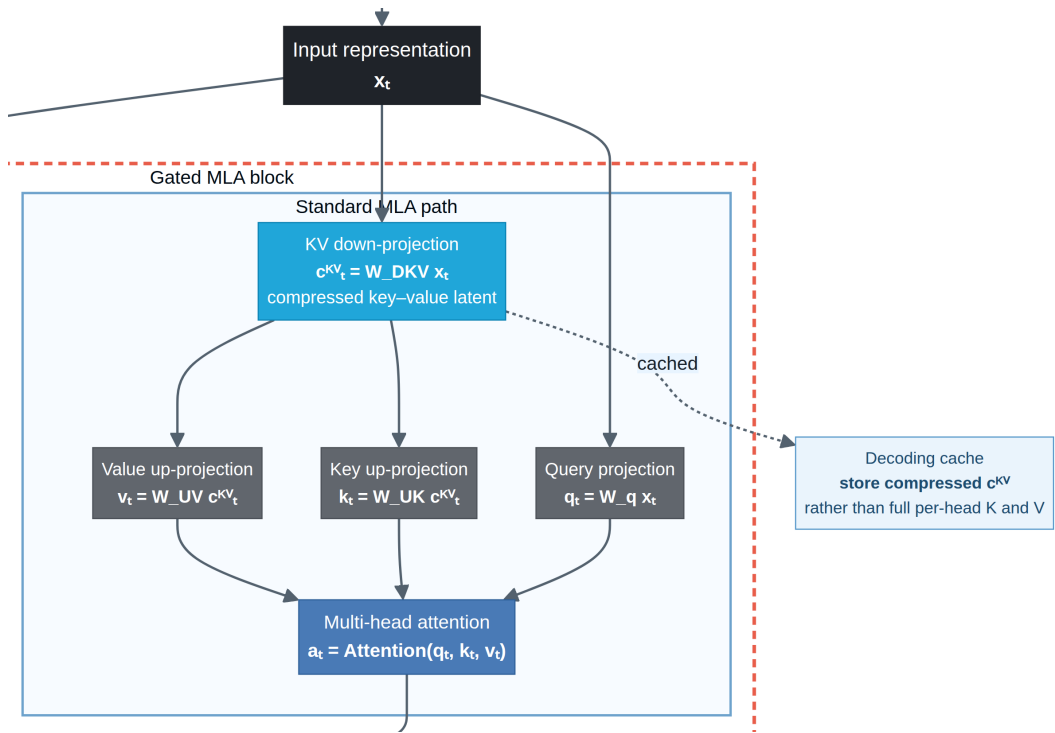


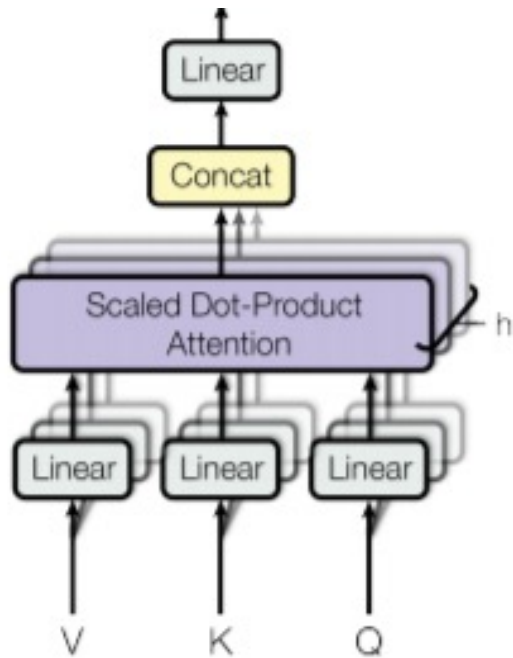
Figure 7: (a) The prefilling time of MLA (full attention), hybrid GDN-H and our Kimi Linear. (b) The time per output token (TPOT) for MLA, GDN-H and Kimi Linear during decoding. (We use batch size = 1 here for tests.)

MLA: Multi-head latent attention

Compressed key-value cache memory

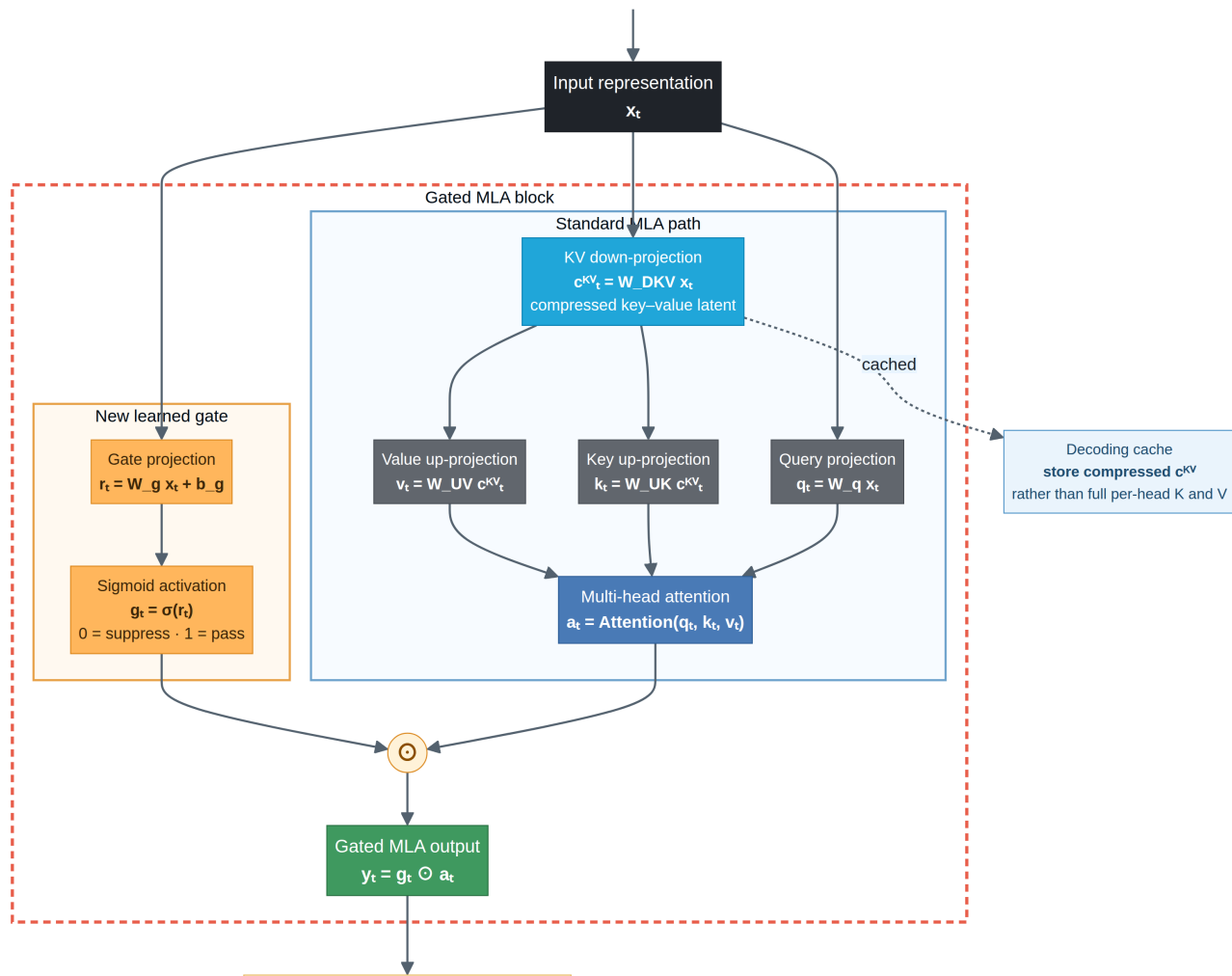


Multi-head attention



Gated MLA

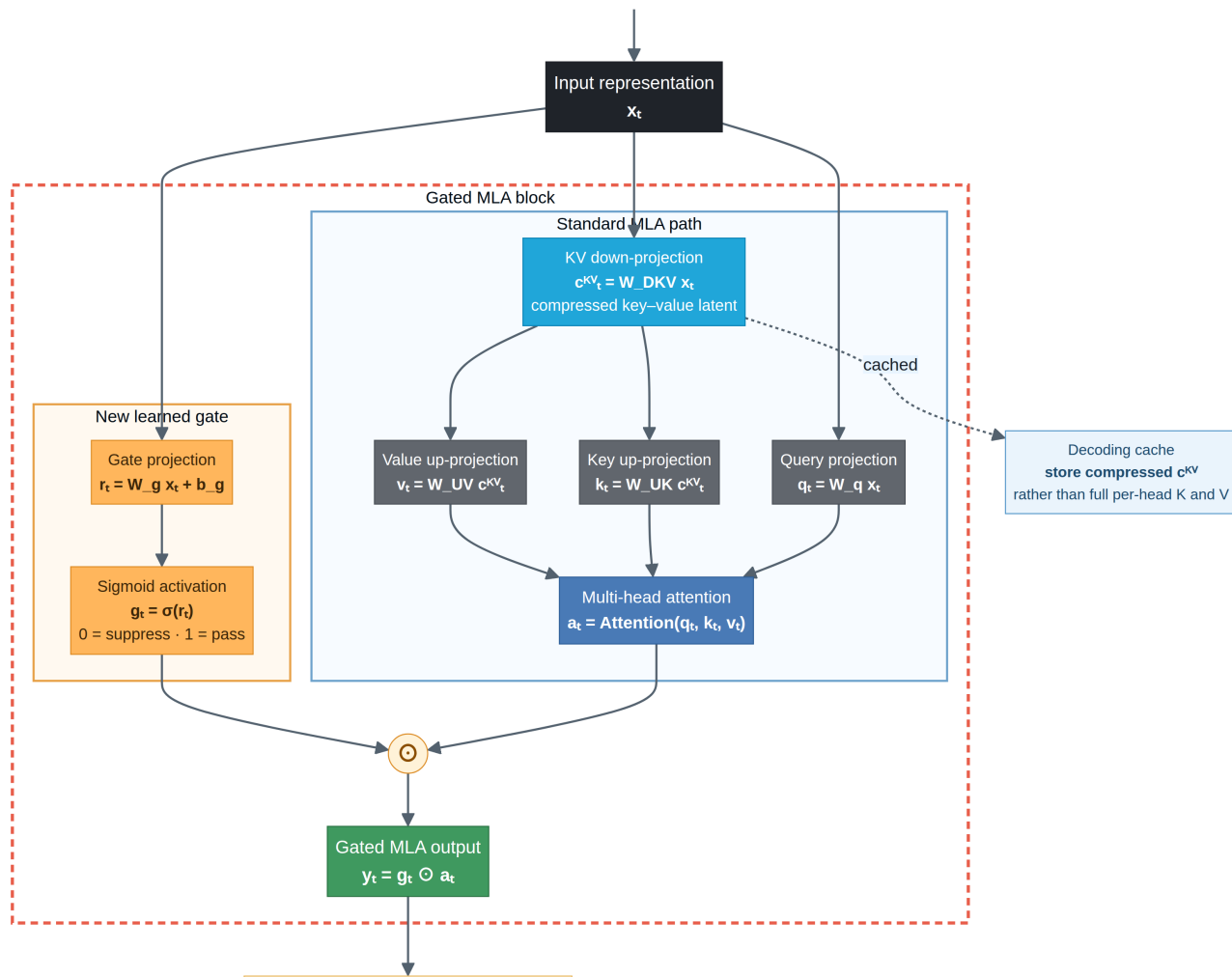
Compressed token
memory with learned
feature selection



Gated MLA

Compressed token
memory with learned
feature selection

Does MLA increase
inference latency?

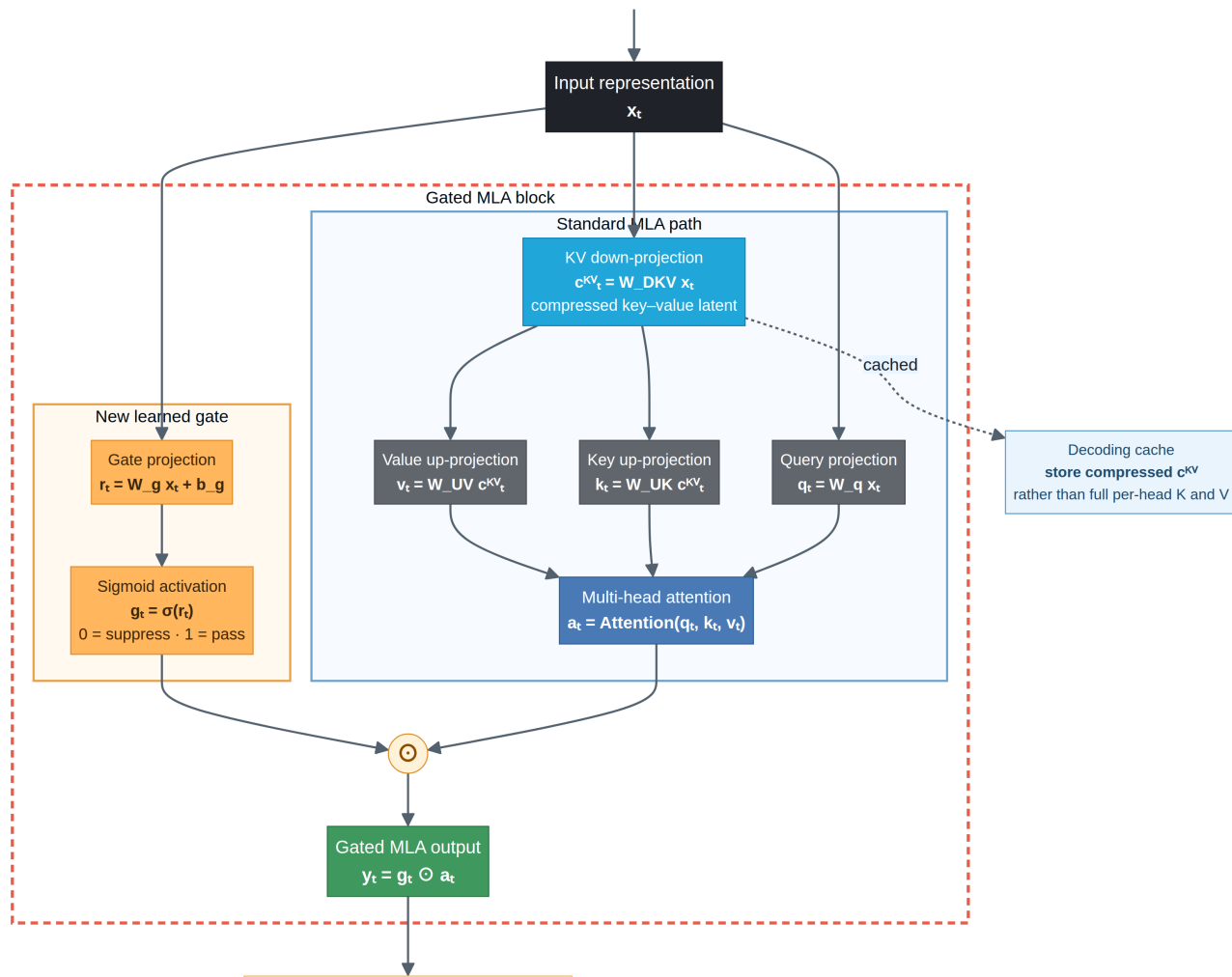


Gated MLA

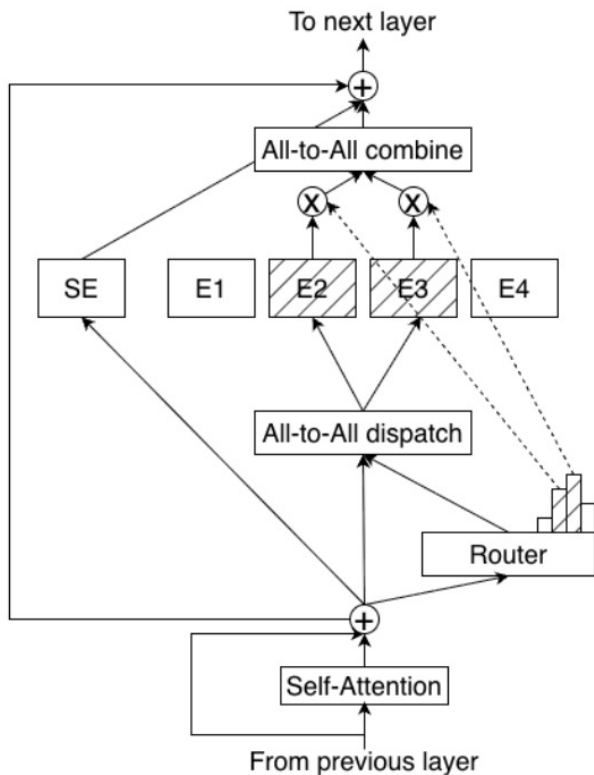
Compressed token
memory with learned
feature selection

**Does MLA increase
inference latency?**

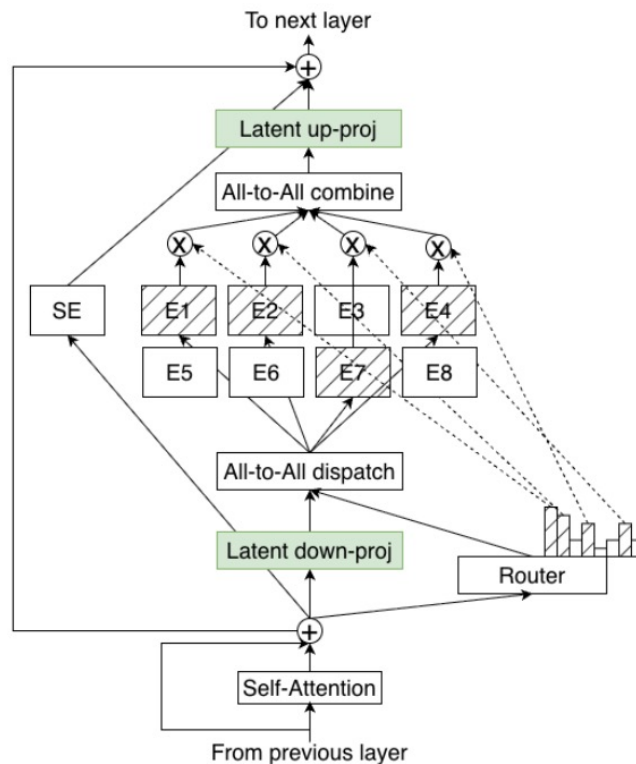
The massive reduction
in KV cache size
eliminates memory
bandwidth bottlenecks,
making MLA faster
overall.



LatentMoE scales expert specialization



(a) Standard MoE architecture.



(b) LatentMoE architecture.

Attention Residuals

Attention Residuals allow each layer to **select useful predecessors** with **learned attention weights**.

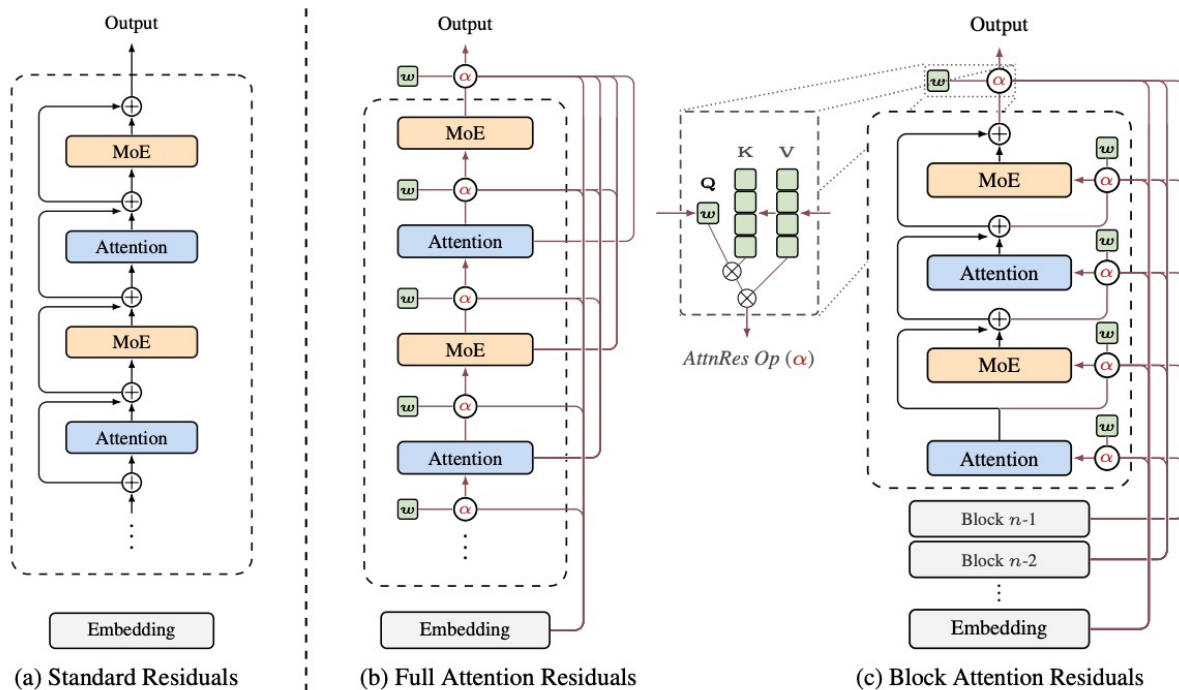


Figure 1: Overview of Attention Residuals. **(a)** Standard Residuals: standard residual connections with uniform additive accumulation. **(b)** Full AttnRes: each layer selectively aggregates all previous layer outputs via learned attention weights. **(c)** Block AttnRes: layers are grouped into blocks, reducing memory from $O(Ld)$ to $O(Nd)$.

Native Multi-Modality

K3 processes text and images within the same model, allowing an agent to inspect the visual result of its own work before it acts again. Native vision broadens the agent's feedback loop.

K3's vision encoder is **MoonViT-V2 (401M parameters)**.

Attribute	Specification
Parameters	~0.4B
Architecture	27-layer ViT · 1024 hidden · 12 heads · 1536 QKV dim
Patch Size	14 × 14
Token Compression	2×2 pixel-shuffle + temporal pooling
Max Resolution	~3584 × 3584
Initialization	From scratch
Source Model	moonshotai/Kimi-K3

K3 Architecture

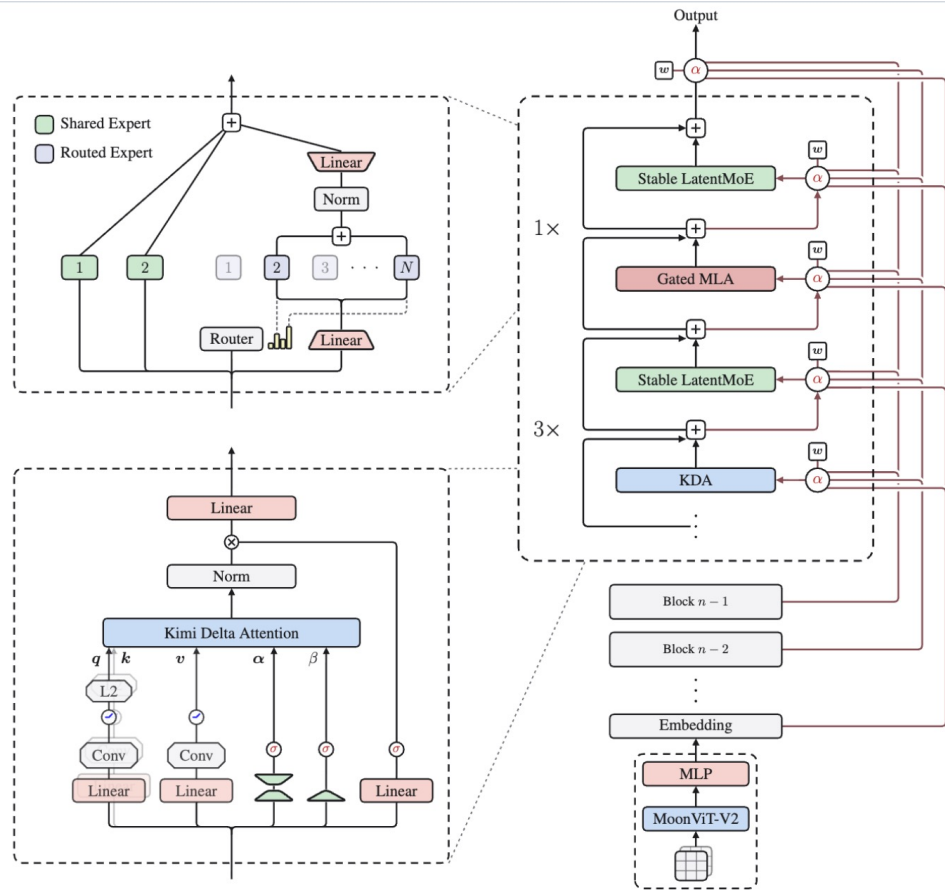


Figure 2: The Kimi K3 architecture, organized around token, channel, and layer mixing, with a native vision pathway at the input. Each block contains three Kimi Delta Attention (KDA) layers followed by one Gated MLA layer, with each attention layer paired with a Stable LatentMoE feed-forward network. Attention Residuals (AttnRes) use learned pseudo-queries (w) to derive attention weights (α) over the embedding and preceding block outputs, enabling selective information flow across depth. **Top left:** the Stable LatentMoE module with shared and routed experts. **Bottom left:** the KDA module. **Bottom right:** the native vision pathway.

Multiple reasoning budgets

The technical report frames **reasoning effort** as a second scaling axis:

K3 is post-trained across general reasoning, coding, and agentic tasks at multiple levels of test-time computation.

Effort level	Intended behavior	Suggested budget dimensions
Low	Answer directly; perform only essential verification.	Thinking tokens, one or few tool calls, short trajectory.
High	Plan, check key assumptions, use tools when useful, and verify.	More thinking tokens, moderate tool-call/turn budget.
Max	Pursue difficult long-horizon work with deliberate verification and recovery.	Largest capped token, tool, turn, and wall-clock budgets.

One deployed model, many trained behaviors: domain- and effort-specialized policies are consolidated through multi-teacher on-policy distillation, targeting trajectories that can span hundreds or thousands of tool calls.

Training: Long-horizon agent training

Million-token agents require stateful reinforcement learning. The problem is larger than retaining a long transcript. An agent must preserve the model state and the task environment while it reasons, acts, and recovers across long episodes.



Build environments before scaling horizon

Curriculum stage	Typical horizon	Environment design	Primary training behavior
Instrumental episodes	A few to a few dozen actions	Small deterministic sandboxes; narrow tool set; immediate checks.	Correct tool calling, observation reading, and stopping after success.
Multi-step projects	Dozens to hundreds of actions	Code repositories, research workspaces, structured office simulations, or visual-reasoning notebooks.	Decomposition, state tracking, testing, and local repair.
Persistent workflows	Hundreds to thousands of actions	Worlds evolving across simulated days; cross-application dependencies and delayed events.	Memory, scheduling, handoffs, recovery, and avoidance of premature completion.
Open-ended execution	Very long trajectories / large context	High-fidelity isolated environments with sparse, independent terminal verification.	Adaptive planning, hypothesis testing, and robust completion under uncertainty.

System Co-Design

At 2.8T parameters and million-token trajectories, KDA, MoE routing, and agent behavior only become practical when the **training and serving stack is built around them**.

KDA systems

Fused kernels, KDA context parallelism, and state-aware prefix caching make the long-sequence attention design efficient within devices, across devices, and across requests.

MoE training systems

MoonEP targets perfectly balanced expert execution with static computation shapes and zero-copy communication, paired with memory-efficient training.

Agent-state systems

External KV retention and resumable sandbox state support long-lived agentic reinforcement-learning rollouts.

Production serving

Specialized kernels plus cache- and budget-aware fleet scheduling aim to turn the model's capabilities into predictable production behavior.

Interpretation: K3's claimed scaling efficiency is not an attention-only or MoE-only result; it is the output of System co-design across the full model lifecycle.

Deployment optimization

K3 is open-weight, but its **2.8T-parameter scale** means self-hosting is an accelerator, networking, and serving-systems decision—not a lightweight model download.

01

Quantized for compatibility

K3 uses quantization-aware training with **MXFP4 weights** and **MXFP8 activations**, designed to broaden hardware compatibility while retaining the model's serving path.

02

Sized for a supernode

Moonshot recommends deployment on supernode configurations with **64 or more accelerators** to make K3 inference efficient at scale.

03

Served by specialized engines

The official repository recommends **vLLM**, **SGLang**, or **TokenSpeed** for inference.

Open weights expand control, but hosted API evaluation is the pragmatic starting point for most teams.

Get Started

<https://openrouter.ai/moonshotai/kimi-k3#providers>

Provider ↑↓	↑↓ Input /M	↑↓ Output /M	↑↓ Cache read /M	↑↓ Latency	↑↓ Throughput	↓ Uptime
 Baseten 	\$3.00	\$15.00	\$0.30	0.73s	86 tps	 100.00%
 Fireworks 	\$3.00	\$15.00	\$0.30	2.43s	41 tps	 100.00%
 Chutes 	\$3.00	\$15.00	\$0.30	1.76s	18 tps	 100.00%
 DigitalOcean 	\$2.85	\$14.25	\$0.285	2.24s	27 tps	 99.84%
 Modal 	\$3.00	\$15.00	\$0.30	1.36s	94 tps	 99.68%
 Moonshot AI 	\$3.00	\$15.00	\$0.30	4.17s	25 tps	 99.46%
 Together 	\$3.00	\$15.00	\$0.30	4.27s	40 tps	 99.24%
 Morph Fast 	\$6.00	\$22.50	\$0.60	4.40s	21 tps	 99.09%
 Phala 	\$3.00	\$15.00	\$1.50	3.16s	31 tps	 98.74%

Config Claude Code with K3

```
export ANTHROPIC_BASE_URL="https://api.moonshot.ai/anthropic"  
export ANTHROPIC_AUTH_TOKEN="${YOUR_MOONSHOT_API_KEY}"  
export ANTHROPIC_MODEL="kimi-k3[1m]"  
export ANTHROPIC_DEFAULT_OPUS_MODEL="kimi-k3[1m]"  
export ANTHROPIC_DEFAULT_SONNET_MODEL="kimi-k3[1m]"  
export ANTHROPIC_DEFAULT_HAIKU_MODEL="kimi-k3[1m]"  
export ANTHROPIC_DEFAULT_FABLE_MODEL="kimi-k3[1m]"  
export CLAUDE_CODE_SUBAGENT_MODEL="kimi-k3[1m]"  
export CLAUDE_CODE_AUTO_COMPACT_WINDOW="1048576"  
export CLAUDE_CODE_EFFORT_LEVEL="max"  
claude
```

enter /status to confirm the configuration

Note: Claude Code uses different model tiers for different scenarios (main conversation, background summarization, sub-agents, and so on). Configuring only some of the variables makes the corresponding scenarios fail silently.

Thank you

Subscribe my latest presentations @ YanAITalk



YanAITalk

@yanaitalk • 8.4K subscribers • 92 videos

Get empowered with AI! ...more

<https://www.youtube.com/@yanaitalk>



Going Houston Machine Learning Meetup

vLLM

LLM Inference & Serving

See how vLLM makes LLM serving

Manage

vLLM: LLM inference and serving

Fri, Sep 4 · 2:00 PM CDT · Online

Online

This session introduces vLLM, an open-source library for LLM inference and serving. We will build an intuitive systems-level understanding of how vLLM...